CYB333 Log Monitor

Security Log Parsing and Console-Based Alerting System | Revised documentation

Repository: https://github.com/Kai00-pen/CYB333_Log_Monitor

Documentation credit: Original README prepared by Ashley Suarez; updated to reflect the later Python detection and testing work. This revision retains the original project explanations and organization.

Project overview

This project uses Python to read a text file of security events and organize each entry so it is easier to review. The sample file records login attempts, an authentication failure, and a blocked network connection. Rather than showing the information as one long line, the program labels the time, event, username, IP address, and result for each entry.

Updated functionality: In addition to parsing and displaying logs, the program now identifies failed **LOGIN** events, prints individual console alerts, counts failed logins by IP address, and displays a warning when an IP has three or more failed LOGIN attempts. It reads a saved sample file; it does not monitor a live system, confirm attacks, or send email or network notifications.

Project files

log_parser.py - Python script for reading, parsing, displaying, and analyzing log entries.
sample_logs.txt - Seven sample security events used to test the program.
README.pdf - Instructions and explanation of the project (this revised document can replace the earlier PDF).

How the sample logs are organized

Each line uses the same five-part format, with a vertical bar between fields:

```
2026-10-05 10:17:03 | LOGIN | admin | 192.168.1.25 | FAILED
```

The date and time tell us when the event occurred. LOGIN is the type of event, admin is the account involved, the IP address identifies the network address recorded with the attempt, and FAILED indicates an unsuccessful login. This consistent layout allows Python to split the entry into useful pieces.

Set up and run the project

1. Install Python from python.org if needed. Check it using **python --version**; some systems use **python3** instead.
2. Download the repository using GitHub Code > Download ZIP and extract it, or clone it with Git:

```
git clone https://github.com/Kai00-pen/CYB333_Log_Monitor.git
```

3. Open the project folder in Visual Studio Code. Confirm that **log_parser.py** and **sample_logs.txt** are both present.
4. Open Terminal > New Terminal in VS Code, navigate to the project folder, and run:

```
python3 log_parser.py
```

On some systems, use **python log_parser.py** or **py log_parser.py** instead.
5. The terminal should print the processed security logs, followed by failed-login detection and suspicious IP detection results.

How the Python script works

1. Reading the file: read_logs()

```
def read_logs(filename):
    logs = []
    try:
        with open(filename, "r") as file:
            for line in file:
                line = line.strip()
                if line:
                    logs.append(line)
    except FileNotFoundError:
        print(f"Error: {filename} was not found.")
    return logs
```

The function starts with an empty list, opens the requested file in read mode, removes extra spaces and line breaks using strip(), skips empty lines, and appends remaining entries. If the file is missing, it prints an error rather than raising an uncaught FileNotFoundError. Finally, it returns the list of entries.

2. Separating the fields: parse_logs()

```
parts = log.split(" | ")
if len(parts) == 5:
    log_entry = {
        "timestamp": parts[0], "event": parts[1],
        "username": parts[2], "ip_address": parts[3],
        "status": parts[4]
    }
    parsed_logs.append(log_entry)
```

The separator divides each log into five parts. Python stores them in a dictionary with named fields. For example, parts[2] becomes the username. Entries with an incorrect number of fields are skipped. This basic parser does not validate IP addresses or timestamps, so five fields alone do not guarantee a correct entry.

3. Printing the results: display_logs()

```
print("=== Processed Security Logs ===")
for log in parsed_logs:
    print(f"Time: {log['timestamp']}")
    print(f"Event: {log['event']}")
    print(f"Username: {log['username']}")
    print(f"IP Address: {log['ip_address']}")
    print(f"Status: {log['status']}")
    print("-" * 40)
```

This function loops through processed entries and prints each field on a separate line. The f-strings insert stored values into the labels, and hyphens separate events for easier review.

4. Detecting failed logins: detect_failed_logins()

```
if (log["event"].strip().upper() == "LOGIN" and
        log["status"].strip().upper() == "FAILED"):
    failed_count += 1
    print("[ALERT] Failed login detected!")
```

This function checks both event type and status, prints an alert with the username, IP address, and timestamp for each matching entry, and prints the total. A failed AUTH event is not counted as a failed LOGIN event.

5. Detecting suspicious IPs: detect_suspicious_ips()

```
ip_counts = {}
# For each failed LOGIN event:
```

```
ip_counts[ip] = ip_counts.get(ip, 0) + 1
# After counting:
if count >= 3:
    print(f"[WARNING] Suspicious IP detected: {ip}")
```

The function groups failed LOGIN attempts by IP address. If an IP reaches three or more failed attempts across the supplied file, it prints a warning about possible brute-force activity. If no IP meets the threshold, it prints "No suspicious IP addresses detected." This is a basic threshold rule, not proof of an attack; it does not apply a time-window restriction.

6. Putting the steps together: main()

```
def main():
    filename = "sample_logs.txt"
    logs = read_logs(filename)
    parsed_logs = parse_logs(logs)
    display_logs(parsed_logs)
    detect_failed_logins(parsed_logs)
    detect_suspicious_ips(parsed_logs)
    return parsed_logs

if __name__ == "__main__":
    main()
```

The main function runs the steps in order and returns the parsed list for possible reuse. The final if statement runs main() when the script is launched directly, rather than imported.

What the sample results show

The seven supplied entries include two successful logins, four failed-status events, and one blocked connection. Three failed LOGIN events involve admin from 192.168.1.25 at 10:17:03, 10:17:15, and 10:17:29 on October 5, 2026 - three failures within 26 seconds. A separate failed AUTH event involves jdoe, and a CONNECTION event is blocked.

Observed output after the update: the program reports **3 failed LOGIN events** and warns that **192.168.1.25** reached the threshold of **3**. The failed AUTH event is displayed in the parsed logs but excluded from the failed LOGIN count. The results justify investigation, not a conclusion that an attack occurred.

Tests and evidence

Completed positive test: With the threshold set to 3 and the original sample data, the program displayed three failed-login alerts and a suspicious-IP warning for 192.168.1.25. Evidence: **CYB333_Suspicious_IP_Detection_Test.png** (suggested screenshot filename).

Completed threshold-negative test: Temporarily changing the threshold to 5 while leaving the sample data unchanged produced "No suspicious IP addresses detected." The total failed LOGIN count remained 3. Evidence: **CYB333_No_Suspicious_IP_Test.png** (suggested screenshot filename). The threshold was restored to 3 afterward and the warning reappeared.

Additional tests to try (not documented as completed): Add a correctly formatted entry and confirm it appears; temporarily rename sample_logs.txt to check the missing-file message; add a malformed entry with fewer than five fields and verify it is skipped. Restore the sample file after each test.

```
2026-10-05 10:30:00 | LOGIN | testuser | 192.168.1.50 | FAILED
```

Common problems

If Python is not recognized, check the installation and try python3 or py. If the log file cannot be found, confirm that the terminal is running in the folder containing sample_logs.txt. If entries are missing, inspect the separators and make sure each line has five fields. Save changes before rerunning. If the suspicious-IP warning is absent, check whether the configured threshold is higher than the count in the sample data.

Possible next steps

Future improvements could include counting within a defined time window, monitoring logs as they arrive, validating IP addresses and timestamps, reporting malformed lines instead of silently skipping them, and sending notifications beyond the terminal. These are proposed extensions, not features claimed for the current project.

Closing notes

This project demonstrates file handling, loops, lists, dictionaries, conditions, functions, basic error handling, log parsing, threshold-based detection, and console alerts in one Python script. It turns plain-text log entries into organized information that can be reviewed for potential security concerns.